Bash command construction and the xv6 shell

Tacu Igor, FAF 242
Operating Systems, Technical University of Moldova
Reproduction performed on 9 November 2026

What this report establishes

The central question in these two labs is where a command line acquires its meaning. An argument inspector receives ordinary strings; it does not know whether a string originally contained quotes, a wildcard, or a variable. By running the same inspector under Bash and xv6, I can attribute the different argument arrays to the shell rather than to the inspected program. The other experiments follow the same method: select a small input, predict the observable effect, and check both a successful and a failing case.

The evidence comes from a new Ubuntu 24.04 ARM64 container on the author's computer and from a newly built xv6 image in QEMU. The blocks below are formatted excerpts from saved console transcripts. The complete commands, source files, and unedited output are committed in the repository.

Lab 3 How Bash selects and expands a command

`type` identified `cd` and `echo` as shell builtins, whereas `ls` was an external executable. This distinction explains why changing directories in a child would not change the shell's own working directory. `command -v ls` supplied a resolved command location; printing PATH one component per line exposed the search order instead of treating PATH as an opaque string. A nonexistent command returned status 127. That status describes failure to locate an executable, not a normal program returning a business result.

Completion and history were tested through a real pseudo terminal running Bash with readline. The input `cd /usr/sh` followed by Tab completed to `/usr/share/`, and `pwd` confirmed the destination. Reverse search selected the previous `pwd` command; executing the selection printed the directory again. The raw PTY recording retains terminal control characters in `evidence/interactive-bash.txt`, so it can be distinguished from a hand-written transcript. The ordinary commands and all subsequent outputs are in `evidence/linux.txt`.

For pathname expansion I created `alpha.c`, `beta.c`, `entry1.txt`, `entry2.txt`, and `two words.txt`. An unquoted `*.c` became two separate names at that point in the run. Quoting the pattern retained the literal asterisk. The unmatched `*.absent` pattern also remained literal because the Bash session had its default unmatched-pattern behavior. These are different reasons for the same visible string: quoting disables expansion, while an unmatched unquoted pattern survives an attempted expansion.

The file containing a space demonstrated a second transformation. `ls 'two words.txt'` addressed one pathname, but `ls two words.txt` passed two independent operands and failed. Brace expansion produced the integers 2 through 5, arithmetic expansion evaluated nine times seven to 63, and single quotes protected `$HOME` while double quotes allowed its value `/root`. None of these transformations requires the invoked executable to implement a pattern or variable parser.

Lab 3 Descriptors status and sequencing

The redirection experiment deliberately combined a valid directory and a missing operand. With stdout and stderr sent to different files, the listing and diagnostic separated, while the command still returned failure. Redirecting stdout first and duplicating descriptor 2 from descriptor 1 put both streams in the same file. Reversing the operations left the error connected to the original stdout destination and made the newly created stdout file empty. Redirections are applied in order to the current descriptor bindings; they are not a declarative request to merge all future output.

The operator tests covered `false && ...`, `false || ...`, and a semicolon after `false`. Only the OR fallback and the command after the semicolon executed. This made the status dependency visible without relying on the behavior of a filesystem command. A background `sleep` was listed with `jobs`, terminated with `kill`, and collected with `wait`; the observed status 143 corresponds to termination by signal 15 in Bash. Saving `$!` immediately after starting the job avoided confusing a later background process with this one.

Lab 3 A small log analysis with explicit inputs

The test log is synthetic: 500 numbered records, seven user identifiers, four page identifiers, and a failure on every fifth record. Its date is a fixture value rather than evidence of real web traffic. This deterministic construction allows a separate check of the pipelines. There must be 100 failures, and each page must receive 125 requests. The nonuniform split between users comes from cycling modulo seven across a total that is not divisible by seven.

`linux/oneliners.sh` answers the eight questions with field-based operations. In particular, it tests `$4 == "FAIL"` rather than accepting an arbitrary substring anywhere on a line. It keeps every user tied for the maximum rather than choosing whichever one happens to sort first. Sorting after aggregation makes ties stable across awk implementations. The relevant observed output was:

```
Q1 failed requests
100

Q2 distinct pages
4 distinct pages
/page0
/page1
/page2
/page3

Q3 requests per page
125 /page0
125 /page1
125 /page2
125 /page3

Q4 most failures
user3 15
user5 15

Q5 user3
user3: 72 requests, 15 failed

Q6 last three failures
10:10 user0
10:15 user5
10:20 user3

Q7 login shells
     20 /usr/sbin/nologin
      2 /bin/bash
      1 /bin/sync

Q8 directory listing line counts
116
117
```

The reusable `linux/report.sh` validates the argument count and readability before invoking awk. One

pass counts total records, failed records, and failures per user. I also supplied a one-record all-OK log, which returned zero failures for every user, and a missing path, which returned status 1. These cases matter because a script can appear correct on the standard fixture while mishandling an empty match set or silently reading standard input when an argument is omitted. The revised script prints a usage error for both missing and unreadable input.

Lab 4 Observing the received arguments

The Linux and xv6 inspectors use the same output notation, `argv[index]=[value]`, so the brackets expose argument boundaries. The Linux run directory contained four C files by the time the comparison ran: the two initial fixtures, the inspector source, and `sample.c`. Consequently `./args *.c 'x y'` received six arguments including argv zero. The space-containing operand remained a single value. An explicitly quoted `$HOME` remained literal, whereas an unquoted or double-quoted variable expression could supply the home directory.

Under xv6 the corresponding wildcard and quoting experiment produced:

```
$ args *.c "x y"
argc=4
argv[0]=[args]
argv[1]=[*.c]
argv[2]=["x]
argv[3]=[y"]
$ args $HOME
argc=2
argv[0]=[args]
argv[1]=[$HOME]
$ args 'single quoted'
argc=3
argv[0]=[args]
argv[1]=['single]
argv[2]=[quoted']
```

The xv6 inspector received the literal `*.c` and two pieces containing quote characters. It therefore does not merely use different wildcard matches; the shell lacks the Bash expansion and quote-removal stages. The single-quote experiment likewise became two arguments containing the quote marks. This conclusion is grounded in the argument arrays rather than in how `echo` happens to display its input.

Lab 4 What the small shell does implement

Redirection and pipelines worked in xv6: writing `hello` to `l4note` and reading it back printed the stored line. `(echo alpha; echo beta) | wc` reported two lines, two words, and eleven bytes. The byte total includes both newline characters. Command lists printed `first` then `second`, and a background command returned to the shell after starting its child. These features correspond to the EXEC, REDIR, PIPE, LIST, and BACK command forms in `xv6/user/sh.c`.

The directory test showed a separate limitation. After changing to `l4case`, a bare `ls` failed with `exec ls failed`, but `/ls ..` listed the parent. The binary remained present; the relative pathname no longer named it from the new current directory. Bash normally resolves such a command through PATH. It would be incorrect to attribute this xv6 failure to missing directory contents or to the `ls` program itself.

`cd` is handled in the long-lived shell because it must affect later commands. By contrast, `runcmd` dispatches most work into a child and performs descriptor changes before `exec`. This explains both the persistence of the working directory and the fact that external commands inherit the shell's selected descriptors. The source inspection is reproducible directly from the complete xv6 tree included in this submission.

Lab 4 Sleep validation and the API boundary

The xv6 header exposes `pause(int)`. The added `xv6/user/sleep.c` accepts one nonnegative decimal tick count, checks integer overflow while accumulating it, and returns failure for invalid text. `sleep 2` returned normally. Omitting the operand printed the usage line, and `sleep -1` printed an invalid-count diagnostic. This makes the shell-facing utility responsible for text validation while leaving the kernel interface responsible for the actual pause.

```
$ sleep 2
$ sleep
usage: sleep nonnegative_ticks
$ sleep -1
sleep: invalid tick count
```

Interpretation and reproducibility

The strongest evidence in these labs is the disagreement between identical argument-inspection logic under two shells. It locates expansion in Bash before execution. The redirection experiment locates output routing in descriptor bindings, and the `cd` experiment locates persistent directory state in the shell process. These observations give different explanations for superficially similar failures and are more useful than memorizing a list of shell features.

Linux sources are in `Lab3-4/linux`. The xv6 sources are `xv6/user/args.c` and `xv6/user/sleep.c`; their registrations are in `xv6/Makefile`. `scripts/run-linux.sh`, `scripts/check-interactive-bash.py`, and `scripts/run-xv6.py` regenerate the evidence. The xv6 kernel and shell derive from the MIT teaching OS; upstream provenance and the source revision are recorded in `xv6/PROVENANCE.md`. Output counts tied to `/etc`, PATH, and account configuration describe the container used for this run, not every Ubuntu installation.